SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4308
Name : K Phani Sridhar Yogi
Regno : 192311091

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Servlet Lifecycle

Phase	Method	Purpose	When Called
Initialization	init()	Load configuration, create resources	Once (server startup)
Service	service()	Handle HTTP requests	Every client request
Destruction	destroy()	Clean up resources	Once (server shutdown)

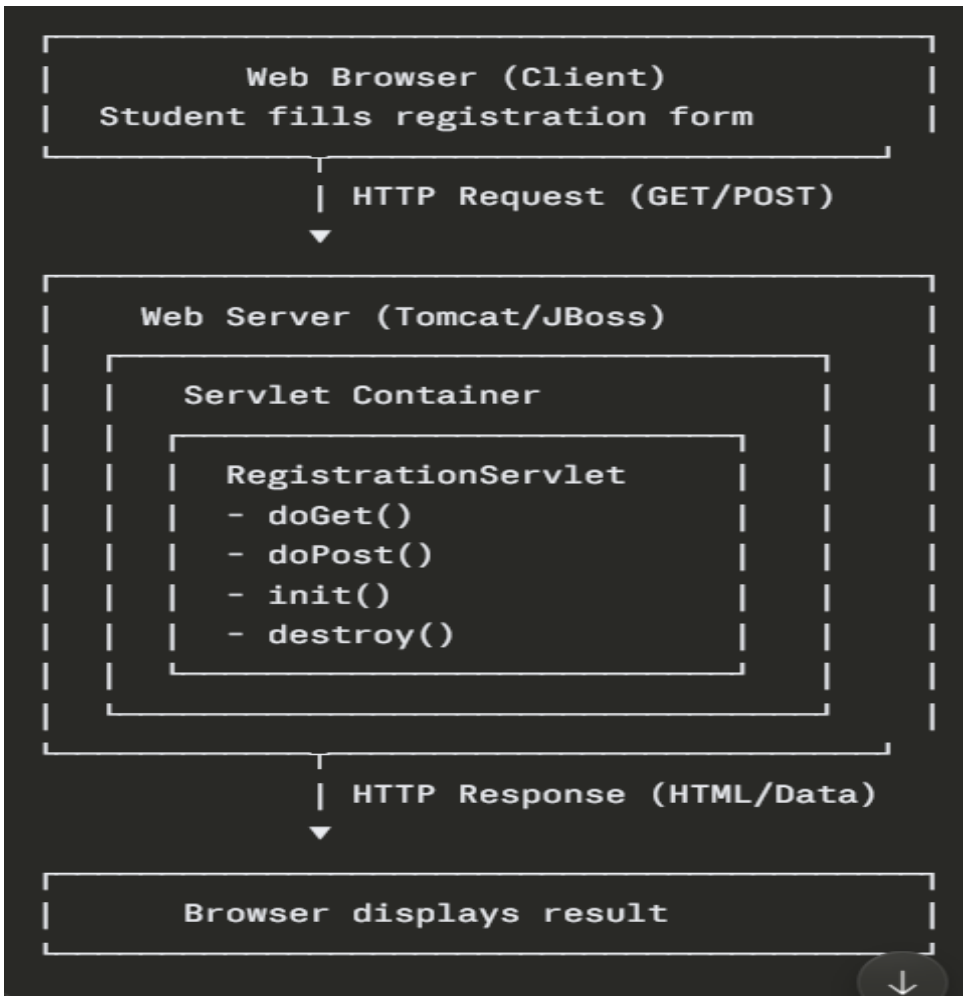
GET vs POST for Sensitive Data

Aspect	GET	POST
Data Location	URL query string	Request body
Visibility	Visible in browser history, logs	Hidden from logs
Security	Unsafe for passwords	Safe for passwords

Aspect	GET	POST
Data Size	Limited (~2KB)	No limit
Semantics	Retrieve data	Submit/modify data

For Registration: **POST is appropriate** because:

- Passwords remain hidden in request body
- Not stored in browser history
- Semantically correct for data submission
- Standard practice for forms




```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class RegistrationServlet extends HttpServlet {
      // Initialization - Called once when servlet loads
   public void init(ServletConfig config) throws ServletException {
      super.init(config);
      System.out.println("Servlet initialized")
   // Handle GET requests (display form)
   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {
```

```
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Student Registration Form</h2>");
        out.println("<form method='POST' action='registration'>");
        out.println("Name: <input type='text' name='name'><br>");
        out.println("Email: <input type='email' name='email'><br>");
        out.println("Password: <input type='password' name='password'><br>");
        out.println("<input type='submit' value='Register'>");
        out.println("</form>");
        out.println("</body></html>");
    }

    // Handle POST requests (process form data)
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get form data
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String password = request.getParameter("password");

        // Process data (validation, database insert, etc.)
        out.println("<html><body>");
        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("</body></html>");
    }

    // Cleanup - Called once when server shuts down
    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

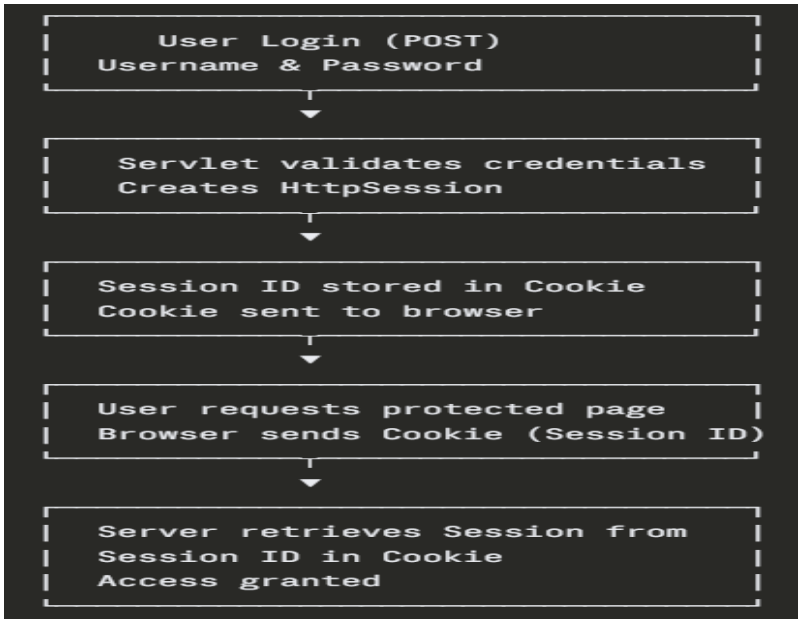
How Session Tracking Works

1. **User logs in** → Server creates HttpSession
2. **Session ID generated** → Stored in Cookie
3. **Cookie sent to browser** → Browser stores it
4. **Subsequent requests** → Browser sends Cookie with Session ID
5. **Server retrieves session data** → Using Session ID from Cookie
6. **User identified** → Access granted across multiple pages

Session vs Cookies Comparison

Feature	HttpSession	Cookies
Storage	Server-side	Client-side (browser)
Security	More secure (data on server)	Less secure (data visible)
Size	Can store large objects	Limited size (~4KB)
Timeout	Server-controlled	Browser-dependent
Use Case	User data, cart, preferences	Remember login, tracking
Example	Login sessions	Remember username

Session is better for sensitive data (login, cart); **Cookies for non-sensitive data** (preferences, tracking).




```
// LoginServlet.java
import javax.servlet.*;
import javax.servlet.http.*;
```

```
import java.io.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Simple validation
        if("admin".equals(username) && "123".equals(password)) {

            // Create session
            HttpSession session = request.getSession(true);

            // Store user data in session
            session.setAttribute("username", username);
            session.setAttribute("loggedIn", true);

            // Set session timeout (30 minutes)
            session.setMaxInactiveInterval(30 * 60);

            out.println("<h2>Login Successful!</h2>");
            out.println("<a href='dashboard'>Go to Dashboard</a>");
        } else {
            out.println("<h2>Login Failed</h2>");
            out.println("<a href='login.html'>Try Again</a>");
        }
    }
}
```

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

JDBC Steps

1. **Load Driver** → Class.forName("com.mysql.jdbc.Driver")
2. **Create Connection** → DriverManager.getConnection(url, user, pass)
3. **Execute Query** → statement.executeQuery() or executeUpdate()
4. **Process Results** → ResultSet rs = statement.getResultSet()
5. **Close Connection** → connection.close()

JDBC Exception Handling

Exception	Cause	Handling
ClassNotFoundException	Driver not found	Add JAR to classpath
SQLException	Query/connection error	Catch and log error
NullPointerException	Connection is null	Check connection validity

Best Practice: Use try-catch-finally or try-with-resources to close connections.

Component	Purpose
Servlet	Handle HTTP requests/responses
POST Method	Secure form data submission
HttpSession	Maintain user login across pages
Cookies	Store Session ID
JDBC	Connect to database, execute SQL
Exception Handling	Catch database errors gracefully

```
// DashboardServlet.java (Protected Page)
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class DashboardServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();

      // Get existing session (don't create new one)
      HttpSession session = request.getSession(false);

      // Check if user is logged in
```

```
        if(session != null && session.getAttribute("loggedIn") != null) {
            String username = (String) session.getAttribute("username");
            out.println("<h2>Welcome, " + username + "</h2>");
            out.println("<a href='logout'>Logout</a>");
        } else {
            out.println("<h2>Please login first</h2>");
            out.println("<a href='login.html'>Login</a>");
        }
    }
}
```

Code of Conduct:

I K Phani Sridhar Yogi 192311091 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand